Descubriendo el código

Grupo 4: RoomsScreen

Aplicación Flutter Hotel Cartagena · Pantalla de consulta de habitaciones disponibles

Integrantes

Rony Valdelamar
Jair Sánchez Pérez
Victor Rocha Ramos
Aníbal Meza Anaya

Archivo analizado: `lib/screens/rooms_screen.dart` (93 líneas)
Archivos relacionados: `services/hotel.dart`, `models/habitacion.dart`, `enums/tipo_habitacion.dart`, `enums/estado_habitacion.dart`, `screens/home_screen.dart`

1. ¿Qué problema resuelve la pantalla?

RoomsScreen responde una sola pregunta del recepcionista: **“¿qué habitaciones tengo libres ahora mismo?”**. Corresponde al requisito funcional **RF03 (Consultar habitaciones)**.

En vez de mostrar una lista larga y desordenada, la pantalla agrupa las habitaciones disponibles por **tipo** (Sencilla, Doble y Suite) y en cada grupo dice cuántas hay libres, el número de cada habitación, cuántas personas caben y su estado. Así el recepcionista puede decidir rápido qué habitación ofrecer antes de ir a la pantalla de reservas.

> Es una pantalla **de solo lectura**: muestra información pero no cambia nada en el hotel.

2. ¿Cómo está construida?

El archivo tiene dos clases, las dos de tipo `StatelessWidget`:

- `RoomsScreen`: la pantalla completa. Recibe el objeto `Hotel` y arma la lista.
- `_EstadoChip`: una etiqueta pequeña de color que muestra el estado de una habitación. El guion bajo (_) significa que es privada: solo se puede usar dentro de este archivo.

Se usa StatelessWidget porque la pantalla no guarda datos propios que cambien mientras está abierta. Cada vez que se abre, el método `build` vuelve a pedirle los datos al Hotel, así que siempre muestra lo más reciente (por ejemplo, después de hacer una reserva).

Árbol de widgets (cómo se meten unos dentro de otros):

```
RoomsScreen
└── Scaffold                          (estructura base de la pantalla)
    ├── appBar: AppBar
    │   └── title: Text('Habitaciones disponibles')
    └── body: ListView                (lista con scroll)
        └── por cada TipoHabitacion (Sencilla, Doble, Suite):
            Card                      (tarjeta)
            └── ExpansionTile         (sección desplegable)
                ├── title: Text('Doble (2 disponible(s))')
                └── children:
                    ├── si no hay → ListTile(Icon info, 'Sin habitaciones disponibles')
                    └── si hay    → ListTile por cada habitación
                                    ├── leading:  Icon(door_front_door)
                                    ├── title:    Text('Habitación 104')
                                    ├── subtitle: Text('Capacidad: 2 persona(s)')
                                    └── trailing: _EstadoChip → Chip('Disponible')
```

Al iniciar la app, la clase Hotel crea 3 habitaciones por tipo: **101–103 Sencilla** (1 persona), **104–106 Doble** (2 personas) y **107–109 Suite** (4 personas). Esas son las que se ven la primera vez.

3. Tipos que se usan y 3 propiedades que los modifican

Para cada tipo explicamos tres propiedades. Las marcadas con ✔ están en el código de RoomsScreen; las marcadas con ○ no se usan en esta pantalla, pero existen en Flutter y sirven para modificar el widget (las incluimos para completar las tres).

Tipo	Propiedad	¿Qué modifica?
Scaffold Esqueleto de la pantalla	✔ `appBar`	Pone la barra superior. Aquí es un AppBar con el título.
	✔ `body`	Define el contenido principal. Aquí es el ListView con las tarjetas.
	○ `backgroundColor`	Cambia el color de fondo de toda la pantalla.
AppBar Barra superior	✔ `title`	Texto que aparece en la barra: 'Habitaciones disponibles'.
	○ `centerTitle`	Si es true, centra el título en la barra.
	○ `actions`	Botones a la derecha de la barra (por ejemplo, un botón de refrescar).
ListView Lista con desplazamiento (scroll)	✔ `padding`	Espacio interno alrededor de la lista: EdgeInsets.all(16) deja 16 píxeles por cada lado.
	✔ `children`	Los elementos de la lista: una Card por cada tipo de habitación.
	○ `scrollDirection`	Dirección del scroll: vertical (por defecto) u horizontal.
Card Tarjeta con bordes redondeados y sombra	✔ `margin`	Espacio por fuera de la tarjeta: EdgeInsets.only(bottom: 16) separa una tarjeta de la siguiente.
	✔ `child`	Lo que va dentro de la tarjeta: el ExpansionTile.
	○ `elevation`	Qué tan marcada es la sombra (la tarjeta parece más “levantada”).
ExpansionTile Sección que se abre y se cierra al tocarla	✔ `initiallyExpanded`	En true hace que la sección aparezca ya abierta; el recepcionista ve todo sin tocar nada.
	✔ `title`	Encabezado de la sección: tipo y cantidad, p. ej. 'Suite (3 disponible(s))'.
	✔ `children`	Lo que aparece al expandir: los ListTile de las habitaciones o el aviso de que no hay.
ListTile Fila estándar de una lista	✔ `leading`	Lo que va a la izquierda: el ícono de puerta (o de información si no hay habitaciones).
	✔ `title / subtitle`	Texto principal ('Habitación 104') y texto secundario ('Capacidad: 2 persona(s)').
	✔ `trailing`	Lo que va a la derecha: el chip de estado (_EstadoChip).
Text Muestra texto	✔ `data (1er parámetro)`	El texto a mostrar. Se arma con interpolación: '$tipo (${habitaciones.length} ...)'.
	✔ `style`	Apariencia del texto. En el título de la sección pone negrita.
	○ `textAlign`	Alineación del texto: izquierda, centro, derecha.

Tipo	Propiedad	¿Qué modifica?
TextStyle Estilo de un texto	✔ `fontWeight`	Grosor de la letra: FontWeight.bold = negrita.
	✔ `color`	Color de la letra. En el chip se usa color.shade700 (tono oscuro del color).
	○ `fontSize`	Tamaño de la letra.
Icon Ícono de Material Design	✔ `icon (1er parámetro)`	Qué ícono se dibuja: Icons.door_front_door o Icons.info_outline.
	○ `size`	Tamaño del ícono en píxeles.
	○ `color`	Color del ícono.
Chip Etiqueta pequeña tipo “píldora”	✔ `label`	Texto del chip: 'Disponible', 'Reservada' u 'Ocupada'.
	✔ `backgroundColor`	Color de fondo: el color del estado con 15 % de opacidad (withOpacity(0.15)), un tono suave.
	✔ `labelStyle`	Estilo del texto del chip: el mismo color en tono oscuro (shade700) para que se lea bien.
EdgeInsets Medidas de espacio (márgenes y rellenos)	✔ `EdgeInsets.all(16)`	El mismo espacio en los 4 lados. Se usa en el padding del ListView.
	✔ `EdgeInsets.only( bottom: 16)`	Espacio solo en el lado indicado. Se usa en el margin de cada Card.
	○ `EdgeInsets.symmetric(...)`	Espacio igual en horizontal y/o vertical.

Tipos de Dart y del modelo del proyecto

Tipo	Propiedad	¿Qué modifica?
Hotel Clase de servicio (lógica del hotel)	✔ `habitaciones`	Lista con todas las habitaciones. De ahí se sacan las disponibles.
	✔ `consultarDisponiblesAgrupadas()`	Devuelve el Map que pinta RoomsScreen.
	✔ `consultarDisponibles()`	Filtra solo las habitaciones con estaDisponible == true.
Habitacion Modelo de una habitación	✔ `numero`	Número de la habitación (101, 102…). Se muestra en el título del ListTile.
	✔ `estado`	Estado actual (EstadoHabitacion). Es la única propiedad que cambia; decide si aparece en la lista.
	✔ `capacidad (getter)`	Personas que caben. No se guarda: se calcula desde el tipo (tipo.capacidadMaxima).
TipoHabitacion enum: sencilla, doble, suite	✔ `values`	Lista con los 3 tipos. RoomsScreen la recorre para crear una tarjeta por tipo.
	✔ `capacidadMaxima`	Capacidad de cada tipo: 1, 2 y 4 personas.
	✔ `toString()`	Texto bonito para la pantalla: 'Sencilla', 'Doble', 'Suite'.

Tipo	Propiedad	¿Qué modifica?
EstadoHabitacion enum: disponible, reservada, ocupada	✔ `disponible`	La habitación está libre. Es la única que aparece en esta pantalla. Chip verde.
	✔ `reservada / ocupada`	Estados después de reservar o hacer check-in. Chip naranja / rojo.
	✔ `toString()`	Texto que se muestra en el chip: 'Disponible', 'Reservada', 'Ocupada'.
Map<TipoHabitacion, List<Habitacion>> Diccionario tipo → lista de habitaciones	✔ `[clave]`	agrupadas[tipo] devuelve la lista de ese tipo.
	✔ `! (null-check)`	Le dice a Dart que la lista nunca es null. Es seguro porque Hotel crea las 3 claves aunque estén vacías.
	○ `keys / length`	Las claves del mapa y cuántas hay.
List<Habitacion> Lista de habitaciones de un tipo	✔ `length`	Cuántas hay. Se muestra en el título: '(3 disponible(s))'.
	✔ `isEmpty`	Si está vacía se muestra 'Sin habitaciones disponibles'.
	✔ `map(...).toList()`	Convierte cada Habitacion en un ListTile y lo vuelve lista para children.
MaterialColor Color de Material con varios tonos	✔ `Colors.green / orange / red`	Color elegido según el estado con un switch.
	✔ `withOpacity(0.15)`	Versión transparente del color (fondo del chip).
	✔ `shade700`	Tono más oscuro del mismo color (texto del chip).

4. ¿Qué información recibe?

Recibe **un solo dato** por el constructor: el objeto `hotel`.

```
final Hotel hotel;
const RoomsScreen({super.key, required this.hotel});
```

`required` obliga a pasarlo; sin él la pantalla no compila. No es un hotel nuevo: es **el mismo objeto** que se creó una sola vez en `main.dart` y que se ha ido pasando de pantalla en pantalla (Login → Home → Rooms). Por eso, si alguien reserva una habitación en otra pantalla, RoomsScreen lo ve.

`_EstadoChip` también recibe un dato: el `estado` de la habitación, para elegir color y texto.

5. ¿Cómo captura información del usuario?

No captura información. No tiene TextField, formularios ni botones de enviar, porque solo sirve para consultar. El usuario solo puede hacer dos cosas:

- Tocar el encabezado de una sección (ExpansionTile) para abrirla o cerrarla. Eso lo maneja el propio widget; no hay código nuestro para eso.
- Hacer scroll en la lista y volver atrás con la flecha del AppBar.

6. ¿Cómo valida los datos?

Como no recibe datos escritos por el usuario, la pantalla **no valida nada por su cuenta**. Esto sigue la regla del proyecto: las pantallas no aplican reglas de negocio; eso lo hace la clase Hotel.

- **El filtro lo hace Hotel:** `consultarDisponibles()` solo deja pasar las habitaciones con `estaDisponible` (estado == disponible). Las reservadas y ocupadas nunca llegan a la pantalla.
- **Lo único que "revisa" la pantalla** es si la lista de un tipo está vacía (`habitaciones.isEmpty`). Si lo está, muestra un aviso en vez de dejar la sección en blanco.
- **El operador ! es seguro** porque `consultarDisponiblesAgrupadas()` crea primero las 3 claves con listas vacías, así que `agrupadas[tipo]` nunca es null.

7. ¿Qué métodos ejecuta?

Paso a paso, cuando se abre la pantalla:

- **1.** Flutter llama a `RoomsScreen.build(context)`.
- **2.** build llama a `hotel.consultarDisponiblesAgrupadas()`.
- **3.** Ese método crea un Map con las 3 claves vacías y llama a `consultarDisponibles()`, que usa `where((h) => h.estaDisponible)` para filtrar.
- **4.** Cada habitación disponible se agrega a la lista de su tipo.
- **5.** De vuelta en build, `TipoHabitacion.values.map(...)` crea una Card por tipo y `habitaciones.map(...).toList()` crea un ListTile por habitación.

- **6.** Para cada fila se usa el getter `h.capacidad` y se construye un `_EstadoChip`, cuyo build usa un `switch` para escoger el color.
- **7.** Los textos se obtienen con el `toString()` de los enums ('Doble', 'Disponible').

8. ¿Cómo se comunica con la clase Hotel?

Por **llamada directa a un método** del objeto que recibió en el constructor: `hotel.consultarDisponiblesAgrupadas()`. La comunicación es **de una sola vía y de solo lectura**: la pantalla pide datos y Hotel responde; RoomsScreen nunca cambia el estado de ninguna habitación ni modifica las listas. Así la lógica queda en un solo lugar (Hotel) y la pantalla solo se encarga de mostrar.

9. ¿Cómo navega hacia otras pantallas?

- **Cómo se llega:** desde HomeScreen, al tocar “Consultar habitaciones”:

```
Navigator.of(context).push(
  MaterialPageRoute(builder: (_) => RoomsScreen(hotel: hotel)),
);
```

- **Cómo se sale:** como la pantalla se abrió con `push`, el AppBar muestra solo la flecha de “atrás”. Al tocarla (o el botón atrás de Android) se hace un `pop` y se vuelve al menú.
- **RoomsScreen no navega a ninguna otra pantalla.** No tiene botones que lleven a Reservar ni a otro lado.

10. ¿Qué ocurre cuando la operación es correcta o incorrecta?

Situación	Qué pasa
Hay habitaciones disponibles (caso normal)	Se ve cada tipo con su cantidad y la lista de habitaciones con capacidad y chip verde “Disponible”.
Un tipo no tiene habitaciones libres	La sección aparece igual, con “(0 disponible(s))” y la fila “Sin habitaciones disponibles” con ícono de información. No es un error, es un aviso.
Se reservó una habitación en otra pantalla	Al volver a abrir RoomsScreen, esa habitación ya no aparece, porque su estado pasó a “reservada”.
Errores (HotelException)	Esta pantalla no tiene try/catch ni usa mostrarError/mostrarExito (SnackBar), porque el método que llama no lanza excepciones: solo lee y filtra. Los mensajes de error y éxito están en las pantallas que sí cambian datos (reservar, check-in, check-out).

Detalle a tener en cuenta: como es StatelessWidget, la lista se calcula cuando se abre. Si los datos cambiaran mientras la pantalla está abierta, no se actualizaría sola; hay que salir y volver a entrar. En esta app no pasa, porque para reservar hay que salir primero de RoomsScreen.

11. ¿Qué modificaciones podrían realizarse?

1. **Mostrar todas las habitaciones, no solo las disponibles.** El _EstadoChip ya soporta los 3 estados (verde, naranja, rojo), pero hoy solo se ve el verde. Se podría usar hotel.habitaciones en vez de consultarDisponiblesAgrupadas() para ver el panorama completo del hotel.
2. **Ir a reservar desde la lista.** Agregar onTap al ListTile para abrir ReserveScreen con el número de la habitación ya escrito.
3. **Botón de refrescar.** Convertir la pantalla en StatefulWidget y poner en actions del AppBar un IconButton que llame a setState(), o usar un RefreshIndicator (deslizar hacia abajo).
4. **Resumen total en el AppBar.** Mostrar en el título el total de habitaciones libres, por ejemplo: “Disponibles: 9”.
5. **Filtro por número de personas.** Un campo o selector para mostrar solo las habitaciones con capacidad suficiente para el grupo del huésped.
6. **Actualizar código obsoleto.** withOpacity() está marcado como obsoleto en las versiones nuevas de Flutter; se reemplaza por color.withValues(alpha: 0.15).

Preguntas para practicar la defensa

P: ¿Por qué RoomsScreen es StatelessWidget y no StatefulWidget?

R: Porque no guarda datos propios que cambien mientras está abierta. Los datos viven en Hotel y se vuelven a leer cada vez que se construye la pantalla.

P: ¿De dónde sale el objeto hotel?

R: Se crea una sola vez en main.dart y se pasa por los constructores: Login → Home → Rooms. Todas las pantallas comparten el mismo.

P: ¿Por qué la pantalla no filtra las habitaciones ella misma?

R: Porque las reglas de negocio son responsabilidad de Hotel. La pantalla solo muestra lo que Hotel le entrega.

P: ¿Por qué siempre aparecen las 3 secciones, aunque un tipo esté vacío?

R: Porque se recorre TipoHabitacion.values (los 3 tipos) y Hotel crea las 3 claves en el Map desde el inicio.

P: ¿Para qué sirve initiallyExpanded: true?

R: Para que las secciones aparezcan abiertas y el recepcionista vea todo de inmediato, sin tener que tocar cada una.

P: ¿Por qué _EstadoChip tiene colores para reservada y ocupada si nunca se muestran?

R: Se dejó genérico para reutilizarlo en el futuro, por ejemplo si se decide mostrar todas las habitaciones.